

SUNIX SDC Expansion Board

CANopen Running via SocketCAN

Application Note

Doc Ver. 2.1

2025 March



SUNIX CO., Ltd.

Tel: +886-2-8913-1987

Fax: +886-2-8913-1986

<http://www.sunix.com>

info@sunix.com.tw

Index

1. Forward	3
2. CANopen Protocol Stack – CANopenNode	3
3. Utility – CANopenLinux	4

1. Forward

CANopen is a High Layer Protocol built on top of the CAN (Controller Area Network). It is commonly used for automation applications in embedded systems. The standards are established by the organization known as CAN in Automation (CiA).

For related information, you can refer to the official website of CiA (CAN in Automation). There are also several third-party tutorial websites available on the internet.

➤ CiA – CANopen:

<https://www.can-cia.org/can-knowledge/canopen>

➤ CSS Electronic – CANopen:

<https://www.csselectronics.com/pages/canopen-tutorial-simple-intro>

➤ National Semiconductor – CANopen:

<https://www.ni.com/en-us/shop/seamlessly-connect-to-third-party-devices-and-supervisory-system/the-basics-of-canopen.html>

2. CANopen Protocol Stack – CANopenNode

There are multiple software vendors in the market that develop CANopen Protocol Stack, e.g. **CANopenMagic**, **CANopenNode**, **CanFesitval** etc. Some of them are paid. However, the purpose of this document is not to introduce various CANopen software. Instead, it aims to explain how our CAN product implements CANopen applications.

CANopenNode is free and open source CANopen protocol stack, it support Linux SocketCAN, that's mean the protocol stack support SUNIX CAN FD Socket driver.

About CANopenNode develop tutorial, please check gihub.io:

<https://github.com/CANopenNode/CANopenNode>

3. Utility – CANopenLinux

CANopenLinux is based on CANopenNode which could running on Linux.

About CANopenLinux tutorial, please check github.io:

<https://canopennode.github.io/CANopenLinux/index.html>

 RUN CANopen Stack with SUNIX CAN by CANopenLinux

➤ SUNIX CAN Channels:

```
root@test-System-Product-Name:~# ip -details link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00 promiscuity 0 minmtu 0 maxmtu 0 addrngenmode eui64 numtxqueues 1 numrxqueues 1 gso_max_size 65536 gso_max_segs 65535
2: eno1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP mode DEFAULT group default qlen 1000
    link/ether 08:bf:b8:a4:9f:35 brd ff:ff:ff:ff:ff:ff promiscuity 0 minmtu 68 maxmtu 9216 addrngenmode none numtxqueues 4 numrxqueues 4 gso_max_size 65536 gso_max_segs 65535 parentbus pci parentdev 0000:05:00.0
    altname enp5s0
3: can0: <NOARP,ECHO> mtu 16 qdisc noop state DOWN mode DEFAULT group default qlen 10
    link/can promiscuity 0 minmtu 0 maxmtu 0
    can state STOPPED (berr-counter tx 0 rx 0) restart-ms 0
    can_sdc: tseg1 1..256 tseg2 1..256 sjw 1..256 brp 1..256 brp-inc 1
    can_sdc: dtseg1 1..64 dtseg2 1..32 dsjw 1..32 dbrp 1..256 dbrp-inc 1
    termination 120 [ 0, 120 ]
    clock 80000000 numtxqueues 1 numrxqueues 1 gso_max_size 65536 gso_max_segs 65535 parentbus platform parentdev can_sdc.4.auto
4: can1: <NOARP,ECHO> mtu 16 qdisc noop state DOWN mode DEFAULT group default qlen 10
    link/can promiscuity 0 minmtu 0 maxmtu 0
    can state STOPPED (berr-counter tx 0 rx 0) restart-ms 0
    can_sdc: tseg1 1..256 tseg2 1..256 sjw 1..256 brp 1..256 brp-inc 1
    can_sdc: dtseg1 1..64 dtseg2 1..32 dsjw 1..32 dbrp 1..256 dbrp-inc 1
    termination 120 [ 0, 120 ]
    clock 80000000 numtxqueues 1 numrxqueues 1 gso_max_size 65536 gso_max_segs 65535 parentbus platform parentdev can_sdc.5.auto
5: can2: <NOARP,ECHO> mtu 16 qdisc noop state DOWN mode DEFAULT group default qlen 10
    link/can promiscuity 0 minmtu 0 maxmtu 0
    can state STOPPED (berr-counter tx 0 rx 0) restart-ms 0
    can_sdc: tseg1 1..256 tseg2 1..256 sjw 1..256 brp 1..256 brp-inc 1
    can_sdc: dtseg1 1..64 dtseg2 1..32 dsjw 1..32 dbrp 1..256 dbrp-inc 1
    termination 120 [ 0, 120 ]
    clock 80000000 numtxqueues 1 numrxqueues 1 gso_max_size 65536 gso_max_segs 65535 parentbus platform parentdev can_sdc.6.auto
6: can3: <NOARP,ECHO> mtu 16 qdisc noop state DOWN mode DEFAULT group default qlen 10
    link/can promiscuity 0 minmtu 0 maxmtu 0
    can state STOPPED (berr-counter tx 0 rx 0) restart-ms 0
    can_sdc: tseg1 1..256 tseg2 1..256 sjw 1..256 brp 1..256 brp-inc 1
    can_sdc: dtseg1 1..64 dtseg2 1..32 dsjw 1..32 dbrp 1..256 dbrp-inc 1
    termination 120 [ 0, 120 ]
    clock 80000000 numtxqueues 1 numrxqueues 1 gso_max_size 65536 gso_max_segs 65535 parentbus platform parentdev can_sdc.7.auto
7: wlp0s20f3: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN mode DORMANT group default qlen 1000
    link/ether 30:05:05:09:48:92 brd ff:ff:ff:ff:ff:ff promiscuity 0 minmtu 256 maxmtu 2304 addrngenmode none numtxqueues 1 numrxqueues 1 gso_max_size 65536 gso_max_segs 65535 parentbus pci parentdev 0000:00:14.3
root@test-System-Product-Name:~#
```

➤ Listening the CAN0

```
root@test-System-Product-Name:/home/test# candump -td -a can0
```

➤ Set CANopen NodeID = 4 on CAN0

```
root@test-System-Product-Name:~# canopen can0 -i 4
canopen[8024]: CANopen device, Node ID = 0x00, starting
canopen[8024]: CAN Interface "can0" RX buffer set to 477 messages (212992 Bytes)
canopen[8024]: CANopen NMT state changed to: "initializing" (0)
canopen[8024]: CANopen device, Node ID = 0x04, communication reset
canopen[8024]: CANopen device, Node ID = 0x04, running ...
canopen[8024]: CANopen NMT state changed to: "pre-operational" (127)
canopen[8024]: CANopen Emergency message from node 0x04: errorCode=0x5000, errorRegister=0x01, errorBit=0x2F, infoCode=0x00000014
^Ccanopen[8024]: CANopen device, Node ID = 0x04, finished
```

➤ CAN0 dump

```
root@test-System-Product-Name:/home/test# candump -td -a can0
(000.000000) can0  704  [1]  00      '.'
(000.050252) can0  084  [8]  00 50 01 2F 14 00 00 00  '.P./....'
```